



Universidad Europea

UNIVERSIDAD EUROPEA DE MADRID
ESCUELA DE ARQUITECTURA, INGENIERÍA Y DISEÑO
INGENIERÍA INFORMÁTICA

PROYECTO DE COMPUTACIÓN II

MADRIVE

MARCELO CID

JUAN DE FRUTOS

MARCOS MARTIN

GERMAN SIERRA

RICARDO VICENTE

Dirigido por

EMILIO YOSHIHIRO ROSANES NISIAMA

CURSO 2025-2026

RESUMEN

La congestión del tráfico urbano sigue siendo uno de los principales problemas de movilidad en las grandes ciudades. La falta de información clara y actualizada sobre el estado real de las vías dificulta la toma de decisiones por parte de los conductores y provoca pérdidas de tiempo, incremento de consumo energético y mayor contaminación. En este contexto, el presente proyecto desarrolla un sistema prototipo capaz de identificar el nivel de tráfico en un tramo concreto de la ciudad de Madrid, mostrando al usuario una salida simple y directa: *“hay tráfico”* o *“no hay tráfico”*.

Se centra en la integración de distintas fuentes públicas de información mediante técnicas de *web scraping* y procesamiento básico de imágenes. El sistema combina estas entradas con un modelo de análisis (basado en YOLO para detección de vehículos) que permite establecer el estado del tráfico en tiempo real o semi real.

ABSTRACT

Urban traffic congestion remains one of the main mobility issues in large cities. The lack of clear and updated information on the actual status of road segments complicates decision-making for drivers and results in time losses, increased energy consumption and higher pollution levels. In this context, this project develops a prototype system capable of identifying the traffic level in a specific area of Madrid, providing the user with a simple and direct output: *“traffic”* or *“no traffic”*.

The work focuses on the integration of public data sources through web scraping techniques and basic image processing. These inputs are combined with an analysis model based on YOLO for vehicle detection, allowing the system to establish the traffic state in real or near-real time.

Capítulo 1. RESUMEN DEL PROYECTO.....	6
1.1 Contexto y justificación.....	6
1.2 Planteamiento del problema.....	6
1.3 Objetivos del proyecto.....	6
1.4 Resultados obtenidos.....	7
1.5 Estructura de la memoria.....	8
Capítulo 2. ANTECEDENTES / ESTADO DEL ARTE.....	9
2.1 Estado del arte.....	9
2.1.1 Visión por Computador: La Hegemonía de los Detectores "Single-Stage".....	9
2.1.2 Modelos Predictivos: De la Estadística al Machine Learning.....	9
2.1.3 Adquisición de Datos: El Reto de los Datos No Estructurados.....	10
2.2 Contexto y justificación.....	10
2.3 Planteamiento del problema.....	11
Capítulo 3. OBJETIVOS.....	12
3.1 Objetivos generales.....	12
3.2 Objetivos específicos.....	12
Capítulo 4. DESARROLLO DE LA APLICACIÓN.....	13
4.1 Planificación del Proyecto.....	13
4.2 Descripción de la solución, metodologías y herramientas empleadas.....	15
4.2.1 Estructura en Tres Capas.....	15
4.2.3 Pipeline Modular.....	16
4.2.4 Criterios de Selección de Datos.....	17
4.2.5 Tecnologías, Lenguajes y Herramientas.....	17
4.2.6 Modelos y Algoritmos Aplicados.....	18
4.3 Recursos Requeridos.....	18
4.4 Presupuesto.....	19
4.5 Viabilidad.....	20
4.6 Resultados del proyecto.....	21
Capítulo 5. TECNOLOGÍAS USADAS.....	22
5.1 Python.....	22
5.2 YOLO (You Only Look Once).....	22
5.3 XGBoost.....	22
5.4 Scikit-learn.....	23
5.4.1 Normalización de datos.....	23
5.4.2 Codificación de variables categóricas.....	23
5.4.3 División del dataset.....	23
5.4.4 Evaluación del modelo.....	23
5.5 Pandas y NumPy.....	24
5.6 Entorno de Gestión de Datos: MariaDB y HeidiSQL.....	24
Capítulo 6. DESCRIPCIÓN DEL PROBLEMA.....	25

Capítulo 7. ANÁLISIS Y DISEÑO DE LA SITUACIÓN.....	26
7.1 Análisis del sistema.....	26
7.2 Requisitos del sistema.....	26
Requisitos funcionales.....	26
Requisitos no funcionales.....	27
7.3 Diseño de la arquitectura del sistema.....	27
Capa de adquisición de datos.....	27
Capa de procesamiento y análisis.....	27
Capa de presentación.....	27
7.4 Diseño de la base de datos.....	28
7.5 Diseño del flujo de datos.....	29
Capítulo 8. IMPLEMENTACIÓN.....	29
8.1 Desarrollo del sistema.....	30
8.2 Implementación del módulo de scraping.....	30
8.3 Implementación del modelo de detección de vehículos.....	30
8.4 Implementación del modelo predictivo.....	31
8.6 Integración con la base de datos.....	33
8.6.1 Tabla usuarios.....	35
Función de la tabla usuarios.....	35
8.6.2 Tabla zonas.....	36
Función de la tabla zonas.....	36
8.6.3 Tabla predicciones.....	36
8.7 Relaciones entre tablas.....	38
8.7.1 Relación entre usuarios y predicciones.....	38
8.7.2 Relación entre zonas y predicciones.....	38
8.7.3 Interpretación global del modelo.....	38
8.8 Claves y restricciones de integridad.....	38
8.9 Justificación del diseño.....	39
8.10 Despliegue del sistema.....	39
Capítulo 9. CONCLUSIONES.....	39
Capítulo 10. GLOSARIO.....	41
Capítulo 11. BIBLIOGRAFÍA.....	42
Capítulo 12. ANEXOS.....	42

Capítulo 1. RESUMEN DEL PROYECTO

1.1 Contexto y justificación

El proyecto tiene como objetivo desarrollar un prototipo de aplicación para el control y predicción de tráfico de la zona entre Avenida Complutense - Paraninfo y Dehesa de la Villa - Avenida de Miraflores.

El sistema utilizará técnicas de web scraping (como con Requests) para recoger datos de tráfico desde fuentes web. Posteriormente, se aplicarán técnicas de análisis de datos y machine learning/deep learning para estimar la congestión, para poder así representar las zonas analizadas en una vista con sus varias cámaras y crear las predicciones deseadas. Todas estas predicciones, junto a las zonas existentes y a los usuarios, se almacenarán en una base de datos para facilitar su uso y conexión.

1.2 Planteamiento del problema

El estado del tráfico en tramos urbanos concretos es altamente variable y depende de factores difíciles de anticipar (horarios pico, eventos locales, condiciones meteorológicas o incidencias puntuales). Esta variabilidad genera incertidumbre en los conductores, quienes carecen de una forma simple de saber, en un momento dado, si una zona estará congestionada o no.

Los datos relevantes para inferir dicha situación existen —cámaras públicas, registros web—, pero no se encuentran integrados ni transformados en un indicador comprensible. El problema a resolver consiste, por tanto, en disponer de un sistema que unifique información heterogénea y la convierta en una respuesta interpretada sobre el estado real del tráfico en un tramo urbano concreto, sirviendo como base para futuras capacidades predictivas.

1.3 Objetivos del proyecto

El objetivo general consiste en diseñar y desarrollar un sistema predictivo de tráfico que integre datos en tiempo real y fuentes externas para estimar el nivel de congestión urbana y evaluar su precisión mediante técnicas de machine learning y deep learning.

1. Implementar un pipeline de scraping basado en la librería Requests para obtener y estructurar datos de tráfico procedentes de fuentes públicas en la web.

2. Diseñar un índice de congestión o estrés vial que permita medir el estado del tráfico combinando información histórica y variables externas relevantes.
3. Entrenar y comparar modelos de aprendizaje automático para predecir el tráfico y evaluar su rendimiento en distintos escenarios.
4. Evaluar la precisión y desempeño del sistema utilizando métricas como MAE (Mean Absolute Error), Accuracy y F-score, con el objetivo de validar la utilidad del modelo propuesto.
5. Representar cada cámara analizada en un mapa de Madrid en los que los usuarios puedan interactuar y seleccionar la zona preferida para crear sus predicciones.
6. (Opcional) Desarrollar un dashboard interactivo que facilite la visualización tanto del estado actual del tráfico como de las predicciones generadas por los modelos.
7. Incorporar una base de datos con la información sobre los varios usuarios, las zonas existentes, y las predicciones que han creado.
8. Crear un sistema de inicio de sesión y creación de cuenta por una pantalla de inicio.
9. Desplegar la aplicación completa en la web.

1.4 Resultados obtenidos

Al haber concluido el proyecto, hemos obtenido un sistema de clasificación y predicción de posibles escenarios y casos de situaciones de tráfico en las calles urbanas e interurbanas de Madrid. El sistema presenta una capa de presentación desarrollada con HTML y JavaScript, basada en un mapa interactivo de Madrid, que permite a los usuarios visualizar distintas cámaras de tráfico distribuidas por la ciudad y seleccionar aquella sobre la que desean realizar una predicción.

El sistema incorpora un modelo de clasificación de Machine Learning basado en Gradient Boosting, entrenado a partir de más de 16000 entradas. Este modelo analiza diferentes variables relacionadas con el tráfico, como la carretera, la fecha y hora, y la latitud y longitud de la cámara, con el objetivo de generar predicciones futuras del nivel de tráfico de una escala de 0 (tráfico nulo) a 3 (tráfico elevado), en distintas ubicaciones de la ciudad.

Además, la aplicación está conectada a una base de datos que gestiona usuarios, zonas y predicciones creadas. Esto permite que los usuarios puedan registrarse e iniciar sesión en la plataforma, seleccionar su cámara de tráfico preferida dentro del

mapa y generar predicciones asociadas a esa ubicación. De esta manera, el sistema no solo permite consultar información de tráfico, sino también crear y almacenar predicciones personalizadas según las necesidades de cada usuario.

El modelo, además de haber sido entrenado con un gran número de datos, es fácilmente escalable a nuevas cámaras de la ciudad o incluso a cámaras de otras ciudades, permitiendo ampliar el sistema a nuevas áreas sin necesidad de modificar significativamente la arquitectura existente.

1.5 Estructura de la memoria

El presente anteproyecto se estructurará en los siguientes capítulos:

- **Capítulo 1:** resumen general del proyecto, incluyendo el planteamiento del problema, los objetivos propuestos, los resultados obtenidos y la descripción de la estructura del documento.
- **Capítulo 2:** introducción, donde se desarrolla el contexto y la justificación del proyecto, así como el estado del arte relacionado con la visión por computador, los modelos predictivos y la adquisición de datos no estructurados, finalizando con el planteamiento del problema.
- **Capítulo 3:** presenta los objetivos del proyecto, abarcando los objetivos generales, los objetivos específicos y los beneficios esperados.
- **Capítulo 4:** planificación del proyecto, desarrollo de la aplicación, detallando el diseño en tres capas, la arquitectura del sistema, las metodologías empleadas, las tecnologías y herramientas utilizadas, los modelos y algoritmos aplicados, así como los recursos requeridos.
- **Capítulo 5:** tecnologías usadas en el desarrollo del proyecto, tanto los varios softwares y sus bibliotecas dedicadas como a hardwares empleados para tareas específicas.
- **Capítulo 6:** descripción del problema que resuelve nuestro proyecto.
- **Capítulo 7:** presenta el equipo de trabajo responsable del desarrollo del anteproyecto.
- **Capítulo 9:** conclusiones de nuestro proyecto, posibles puntos débiles de nuestra estructura, y futuras mejoras.
- **Bibliografía, Glosario y Anexos de la memoria.**

Capítulo 2. ANTECEDENTES / ESTADO DEL ARTE

El proyecto tiene como objetivo desarrollar un prototipo de aplicación para el control y predicción de tráfico de la zona entre Avenida Complutense - Parainfo y Dehesa de la Villa - Avenida de Miraflores.

El sistema utilizará técnicas de web scraping para recoger datos de tráfico desde fuentes web. Posteriormente, se aplicarán técnicas de análisis de datos y machine learning/deep learning para estimar la congestión.

2.1 Estado del arte

La monitorización del tráfico urbano se encuentra en una fase de transición tecnológica, desplazándose desde los sistemas de hardware intrusivo (sensores bajo el asfalto) hacia los **Sistemas de Transporte Inteligente (ITS)** basados en software y visión artificial. El estado actual de la tecnología se define por la convergencia de tres pilares fundamentales que sustentan este proyecto: la detección de objetos en tiempo real, el aprendizaje automático aplicado a datos tabulares y la ingeniería de datos sobre fuentes públicas.

2.1.1 Visión por Computador: La Hegemonía de los Detectores "Single-Stage"

En el ámbito de la detección vehicular, la literatura científica distingue dos arquitecturas principales:

- **Detectores de dos etapas (ej. Faster R-CNN):** Priorizan la máxima precisión a costa de una alta carga computacional, lo que limita su uso en tiempo real.
- **Detectores de una etapa (ej. YOLO - You Only Look Once):** Procesan la imagen en una única pasada de red neuronal.

El estándar actual de la industria para aplicaciones de vídeo en tiempo real es la familia **YOLO**. A diferencia de sus sucesores, modelos como **YOLOv8** logran un equilibrio óptimo entre velocidad (FPS) y precisión (mAP), permitiendo la detección de objetos de diferentes escalas (como vehículos lejanos) mediante redes piramidales de características (FPN), lo que justifica su elección para el procesamiento de imágenes de cámaras de tráfico con perspectivas variables.

2.1.2 Modelos Predictivos: De la Estadística al Machine Learning

Para la estimación de la congestión, el estado del arte ha superado los modelos estadísticos lineales clásicos (como ARIMA), que fallan ante la no linealidad del

tráfico urbano. La tendencia actual favorece el uso de algoritmos de **Ensemble Learning** (aprendizaje por conjuntos).

Modelos como **Random Forest** y, más recientemente, **XGBoost (eXtreme Gradient Boosting)**, se han posicionado como líderes en el manejo de datos estructurados heterogéneos (mezcla de conteos numéricos, variables categóricas temporales y datos meteorológicos). Estos algoritmos ofrecen una mayor robustez frente al sobreajuste y una mejor interpretabilidad que las redes neuronales profundas ("cajas negras") cuando se trata de tareas de clasificación tabular, alineándose con los resultados obtenidos en este proyecto.

2.1.3 Adquisición de Datos: El Reto de los Datos No Estructurados

El paradigma de **Smart Cities** ha impulsado la apertura de datos públicos (*Open Data*), pero a menudo carecen de estructuración o APIs estandarizadas. La ingeniería moderna resuelve esto mediante **pipelines ETL (Extract, Transform, Load)** automatizados. El uso de técnicas de *web scraping* para transformar fuentes no estructuradas (imágenes web, HTML) en datasets estructurados es una práctica consolidada en la ciencia de datos aplicada, permitiendo generar inteligencia histórica en escenarios donde no existen registros oficiales previos.

2.2 Contexto y justificación

La movilidad urbana en grandes ciudades como Madrid se ha convertido en un problema creciente debido al aumento del volumen de vehículos y a la variabilidad de factores externos que afectan al tráfico, las incidencias viales o la distribución horaria de los desplazamientos. Esta complejidad dificulta que el usuario pueda anticipar de forma fiable el estado de circulación en un tramo concreto, generando incertidumbre y limitando la capacidad de planificación.

En la actualidad existen aplicaciones que informan sobre el estado del tráfico, pero la mayoría se limitan a ofrecer datos descriptivos sin realizar una integración inteligente de múltiples fuentes o sin proporcionar predicción basada en patrones históricos. Además, en muchos casos la información presentada es global o demasiado amplia, lo que reduce la utilidad para la toma de decisiones a nivel local o en tramos específicos.

Este proyecto surge como respuesta a esta necesidad, proponiendo un prototipo que combina adquisición automática de datos (mediante scraping) y técnicas de análisis predictivo. Con ello se busca no solo indicar el estado actual del tráfico, sino también anticipar su evolución a corto plazo a partir de patrones reales.

La aportación principal del proyecto reside en la construcción de un sistema predictivo centrado en el análisis de la congestión de distintas zonas de Madrid, especialmente en sus carreteras más importantes. Este enfoque facilita la validación

en un entorno realista y sienta las bases para futuras ampliaciones a la misma zona urbana u otras ciudades.

En términos de aplicabilidad, el prototipo puede servir como base para herramientas orientadas a movilidad inteligente (Smart Mobility), asistencia a la planificación urbana o complementación de datos públicos mediante inteligencia artificial. Al tratarse de una implementación modular, el sistema puede evolucionar desde un entorno académico a un producto tecnológico más complejo, incorporando nuevas fuentes de datos o funcionalidades interactivas en fases posteriores.

2.3 Planteamiento del problema

El estado del tráfico en tramos urbanos concretos es altamente variable y depende de factores difíciles de anticipar (horarios pico, eventos locales, condiciones meteorológicas o incidencias puntuales). Esta variabilidad genera incertidumbre en los conductores, quienes carecen de una forma simple de saber, en un momento dado, si una zona estará congestionada o no.

Los datos relevantes para inferir dicha situación existen —cámaras públicas, registros web—, pero no se encuentran integrados ni transformados en un indicador comprensible. El problema a resolver consiste, por tanto, en disponer de un sistema que unifique información heterogénea y la convierta en una respuesta interpretada sobre el estado real del tráfico en un tramo urbano concreto, sirviendo como base para futuras capacidades predictivas.

Capítulo 3. OBJETIVOS

3.1 Objetivos generales

El objetivo general del presente trabajo consiste en diseñar y desarrollar una aplicación web integral para el control y predicción del tráfico urbano en Madrid, que unifique información heterogénea mediante técnicas de visión artificial y aprendizaje automático, ofreciendo una interfaz usable y accesible tanto para el público general como para usuarios registrados.

3.2 Objetivos específicos

Para alcanzar el objetivo general, se proponen los siguientes hitos técnicos vinculados a los requisitos de la asignatura:

1. **Integración de Datos y Algoritmos:** Evolucionar los modelos de detección vehicular (YOLOv8) y clasificación (XGBoost) desarrollados previamente para su funcionamiento en un entorno web.
2. **Gestión de Base de Datos:** Diseñar e implementar una base de datos estructurada para el almacenamiento de entidades como usuarios, zonas de predicción y registros históricos de tráfico.
3. **Desarrollo de Backend (API):** Implementar una arquitectura de API para la gestión de datos, incluyendo la creación de endpoints funcionales y la gestión de seguridad mediante tokens para el login.
4. **Interfaz de Usuario (Frontend):** Desarrollar una interfaz interactiva (basada en Streamlit o tecnologías web similares) que incorpore librerías de gráficos para la visualización de datos y cumpla con las heurísticas de usabilidad.
5. **Gestión de Roles y Acceso:** Establecer un sistema de control de acceso que diferencie entre el usuario público, usuarios registrados con funciones adicionales y una interfaz de administración restringida.
6. **Despliegue y Producción:** Realizar la packetización de la aplicación para su correcta puesta en producción y despliegue final.

Capítulo 4. DESARROLLO DE LA APLICACIÓN

4.1 Planificación del Proyecto

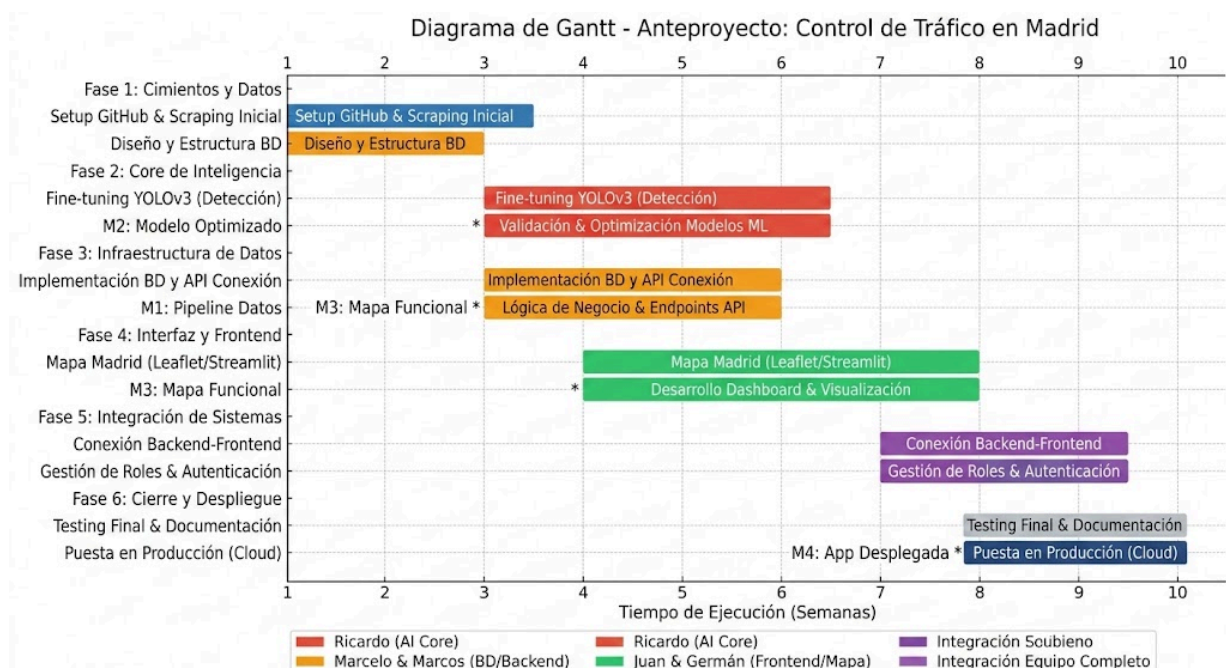
La ejecución del proyecto se divide en fases lógicas que permiten avanzar desde la captura de datos hasta el despliegue de la aplicación web funcional. Se estima una dedicación total de **400 horas** distribuidas equitativamente.

Fase	Tareas Principales	Semanas	Responsables
1. Cimientos y Datos	Configuración de Git/GitHub, Scraping inicial y diseño de DB	1-2	Marcelo, Marcos, Ricardo
2. Core de Inteligencia	Entrenamiento y optimización de YOLOv8/XGBoost	3-5	Ricardo
3. Infraestructura de Datos	Implementación de la BD y API de conexión	3-5	Marcelo, Marcos
4. Interfaz y Frontend	Creación del mapa interactivo y vistas de usuario	4-7	Juan, Germán
5. Integración de Sistemas	Conexión Backend-Frontend y gestión de roles	8-9	Todo el equipo
6. Cierre y Despliegue	Testing final, documentación y puesta en producción	10	Todo el equipo

Para maximizar la eficiencia, se han definido los siguientes flujos de trabajo basados en las especialidades del equipo :

- **Ricardo (Modelos ML/DL):**
 - Refinar el sistema de detección vehicular utilizando YOLOv8 con *fine-tuning*.
 - Optimizar los modelos de clasificación de congestión (XGBoost) para su integración.
- **Marcelo y Marcos (Base de Datos):**
 - Diseñar el esquema para almacenar usuarios, zonas y registros históricos.
 - Asegurar la integridad y eficiencia en las consultas de predicciones en tiempo real.
- **Juan y Germán (Frontend y Mapa):**
 - Desarrollar el mapa interactivo de Madrid con la ubicación de las cámaras públicas.
 - Implementar las interfaces de usuario (login) y el dashboard de visualización con Streamlit.

1. **Hito 1 (Semana 2):** Pipeline de scraping funcional y base de datos inicial creada.
2. **Hito 2 (Semana 5):** Modelo de detección de vehículos optimizado con precisión validada.
3. **Hito 3 (Semana 7):** Interfaz del mapa interactivo capaz de mostrar datos en tiempo real.
4. **Hito 4 (Semana 10):** Aplicación desplegada en la web con acceso diferenciado por roles.



4.2 Descripción de la solución, metodologías y herramientas empleadas

La solución desarrollada consiste en un sistema modular y automatizado para el análisis y predicción del estado del tráfico. El enfoque se centra en la trazabilidad de los datos y la extensibilidad del sistema. Se distinguen tres fases troncales: adquisición de datos, procesamiento y modelado, y visualización de resultados (opcional). El diseño modular garantiza que cada componente (por ejemplo, el módulo de *scraping* o el modelo YOLO) pueda ser mejorado o reemplazado individualmente, asegurando la robustez y la adaptabilidad futura de la plataforma.

La aplicación está diseñada bajo una arquitectura de tres capas bien definida, que garantiza la modularidad, la escalabilidad y una clara separación de responsabilidades. Esta estructura facilita tanto el mantenimiento como la incorporación de futuras funcionalidades, descomponiendo el flujo de trabajo en la adquisición, procesamiento y presentación.

4.2.1 Estructura en Tres Capas

Capa de Adquisición de Datos

Esta primera capa se encarga de la ingesta de información en bruto desde diversas fuentes externas y públicas. Su función principal es el scraping de datos en tiempo real o casi real, incluyendo *feeds* de cámaras de tráfico, reportes de incidencias viales y datos externos complementarios (como información meteorológica). La tarea crucial de esta capa es la estructuración inicial de los datos. Toda la información adquirida, independientemente de su formato original (imágenes, JSON, HTML), se almacena de manera organizada, asegurando que cada registro esté intrínsecamente asociado a una fecha, una hora y una localización geográfica específica. Esto sienta las bases para el posterior procesamiento y análisis.

Capa de Procesamiento y Clasificación

El análisis del sistema se hace en esta capa, donde los datos brutos se transforman en *inteligencia* procesable. El proceso comienza con el preprocesamiento de datos, que incluye tareas esenciales de limpieza (manejo de valores nulos, corrección de errores) y normalización (estandarización de formatos y escalas). La parte más innovadora es la extracción de información de imágenes mediante técnicas de visión artificial. Se emplea un modelo YOLO (en particular la versión 8), ajustado para la tarea de conteo de vehículos en las imágenes de las cámaras. A partir de este conteo, se calcula la ocupación de la calle, proporcionando una medida objetiva y precisa de la densidad vial. Esta capa es responsable de clasificar el estado de la vía (ej. "tráfico lento", "fluido", "congestión") antes de la presentación final.

Capa de Presentación

Esta es la capa orientada al usuario, diseñada para ofrecer una interfaz sencilla e intuitiva. Su objetivo es mostrar el resultado del análisis de manera directa y comprensible. Permite al usuario seleccionar una zona o tramo específico de interés (por ejemplo, mediante un mapa o un menú desplegable). La funcionalidad principal es la visualización directa del resultado, presentando el estado actual del tráfico en el tramo seleccionado, utilizando los indicadores de congestión generados por la capa de procesamiento.

4.2.2 Metodología de Desarrollo Incremental

El proyecto se ha abordado mediante una metodología incremental, que prioriza la validación continua y el paso de un entorno de datos sin tratar a un sistema de predicción automatizado en un entorno controlado. Los pasos clave incluyen:

1. **Recopilación y Selección de Datos:** Fase inicial enfocada en asegurar la **legalidad** de las fuentes y la **estabilidad** de sus URLs públicas.
2. **Procesamiento y Normalización:** Transformación de los datos brutos en un **formato tabular estructurado** listo para el análisis.
3. **Análisis y Creación de Variables Derivadas:** Cálculo de métricas clave como el **índice de congestión** a partir del conteo vehicular y otros *inputs*.
4. **Entrenamiento y Comparación de Modelos:** Aplicación de ML, seguida de una comparación rigurosa de su rendimiento.
5. **Incorporación de frontend:** Codificación de la ventana principal (mapa de Madrid con sus autovías y cámaras) para la visualización de las distintas cámaras y cálculo del nivel de congestión a partir de cada una de ellas.
6. **Incorporación de una base de datos:** Creación de la base de datos para el almacenamiento de entidades (usuarios, zonas de predicciones y predicciones).
7. **Implementación de vistas basadas en roles:** Diferenciación de distintas vistas dependiendo del rol del usuario conectado a la aplicación.
8. **Despliegue de la aplicación en la web.**

4.2.3 Pipeline Modular

El sistema funciona mediante un pipeline modular que organiza el flujo de datos desde su captura hasta la generación de indicadores de tráfico. Esta estructura secuencial y flexible permite añadir nuevas fuentes de datos o actualizar los modelos sin interrumpir el funcionamiento general.

El proceso comienza con la entrada de información: se recogen imágenes de las cámaras, datos de incidencias y variables meteorológicas. A continuación, los datos

pasan por un procesamiento inicial que incluye scraping, extracción de información relevante como el conteo de vehículos con YOLO y la normalización en un formato estructurado. Una vez listos y enriquecidos, los datos alimentan el análisis predictivo, donde se aplican técnicas de Machine Learning, Deep Learning y series temporales para anticipar el estado futuro del tráfico.

El resultado final se presenta al usuario como un indicador claro de congestión y también sirve para alimentar un dashboard de visualización de cada una de las cámaras. De forma paralela, todos los datos procesados y las predicciones generadas se almacenan en una base de datos y se mantienen logs de trazabilidad (IDs y horas/fechas), garantizando la auditoría y la mejora continua del sistema.

4.2.4 Criterios de Selección de Datos

La selección de datos del proyecto se basó en criterios claros que garantizan su relevancia y viabilidad. Se eligieron exclusivamente tramos concretos de Madrid, entre la Avenida Complutense - Paraninfo y Dehesa de la Villa - Avenida de Miraflores, asegurando que los datos fueran directamente aplicables al área de estudio.

Se priorizaron fuentes públicas con URLs estables a largo plazo, lo que permitió automatizar la captura periódica de información. Además, se respetaron estrictamente las condiciones de uso y los límites de acceso de cada fuente, asegurando la legalidad del sistema.

Por último, se dio especial importancia a que cada registro estuviera vinculado a imágenes de cámaras públicas, lo que facilita el uso de visión artificial para obtener medidas objetivas del tráfico, como el conteo de vehículos, evitando depender de observaciones manuales.

4.2.5 Tecnologías, Lenguajes y Herramientas

El proyecto se apoya en un stack tecnológico sólido, centrado en Data Science y Machine Learning con Python. Se utilizó Python 3.10 como lenguaje principal por su ecosistema amplio y maduro de librerías para scraping y aprendizaje automático. Para la captura de datos se emplearon Requests, para gestionar las solicitudes HTTP, complementada con datos en formato JSON que acompañan a las imágenes de las carreteras.

En el procesamiento de datos, pandas y numpy permiten manipular, limpiar y construir el dataset final, mientras que pathlib facilita la gestión de rutas de archivos de manera independiente del sistema operativo y joblib agiliza la serialización y el despliegue de los modelos entrenados. La parte de visión artificial y machine learning se centra en YOLOv8 con fine tuning, que permite contar vehículos con precisión y generar un dataset enriquecido con metadatos como fecha, hora, tramo y fuente de cada observación.

Adicionalmente, se desarrolló una aplicación web que actúa como interfaz principal del sistema. Esta aplicación incorpora una vista principal basada en un mapa interactivo, que permite visualizar las distintas carreteras monitorizadas y acceder a la información asociada a cada tramo. A través de esta vista, el usuario puede seleccionar una carretera concreta y generar predicciones personalizadas del estado del tráfico en función de los datos históricos disponibles y los modelos entrenados.

Para dar soporte a la aplicación web y a la persistencia de la información, se implementó una base de datos destinada a la gestión de usuarios, zonas geográficas y predicciones generadas. Esta base de datos permite almacenar de forma estructurada tanto la información estática (usuarios y carreteras) como los resultados dinámicos derivados de los modelos predictivos, facilitando la consulta, reutilización y escalabilidad del sistema.

4.2.6 Modelos y Algoritmos Aplicados

La metodología para la extracción de la inteligencia del tráfico se realiza mediante dos etapas fundamentales de modelado. La primera fase es la detección de vehículos con YOLO (You Only Look Once), donde el modelo se aplica directamente sobre las imágenes para el conteo de vehículos, proporcionando una medida objetiva y cuantificable de la densidad vial en un momento dado.

La información de volumen vehicular obtenida alimenta la segunda etapa, que es la clasificación del estado del tráfico. En esta fase, el volumen estimado de vehículos se utiliza para realizar una clasificación categórica multiclass que determina el estado del tráfico en una escala discreta, por ejemplo, de 0 a 3, representando distintos niveles de fluidez.

Estos resultados se integran en la aplicación web, permitiendo que las predicciones generadas puedan visualizarse sobre el mapa interactivo y asociarse a cada tramo de carretera. En fases posteriores de desarrollo y de cara a la evaluación final del proyecto, los datos clasificados servirán como base para la aplicación de modelos predictivos avanzados, incluyendo análisis de series temporales y técnicas de Machine Learning o Deep Learning orientadas a la predicción del tráfico futuro.

4.3 Recursos Requeridos

El proyecto pudo llevarse a cabo gracias a una combinación de recursos técnicos, de datos y humanos. En la parte técnica se utilizó **Python 3.10** como lenguaje principal, junto con librerías como **Requests** para el *scraping*, y **pandas**, **numpy**, **pathlib** y **joblib** para el procesamiento de datos. La visión artificial se basó en el modelo **YOLOv8** ajustado mediante *fine tuning*. El desarrollo se realizó con **Visual Studio Code** y el control de versiones con **Git** y **GitHub**.

Para la aplicación web se requirieron recursos adicionales orientados al desarrollo frontend y backend, así como una **base de datos** para la gestión de usuarios, zonas y predicciones. Estos componentes permiten ofrecer una vista principal basada en un mapa interactivo y asegurar la persistencia de la información generada por el sistema.

Los datos proceden principalmente de **cámaras públicas de tráfico en Madrid**, acompañados de metadatos obtenidos de fuentes abiertas y estables, lo que garantiza su uso continuado sin problemas legales. Para el entrenamiento y ajuste de los modelos, especialmente YOLO, se utilizó un equipo local con capacidad de procesamiento suficiente y una conexión a internet rápida, necesaria para la descarga y actualización de información en tiempo real.

En cuanto a los recursos humanos, el proyecto fue desarrollado por el equipo de trabajo descrito anteriormente, con la supervisión académica del profesor responsable, lo que permitió mantener una metodología de trabajo estructurada y rigurosa durante todo el proceso.

4.4 Presupuesto

El equipo está compuesto por cuatro ingenieros: Ricardo, Jorge, Marcos y Germán.

- **Horas estimadas:** 10 semanas x 10 horas/semana por persona = 400 horas totales.

- **Precio/hora (Junior):** 25 €/h.

- **Total Personal:** 400 x 25 = **10.000 €**.

Hardware: Uso de equipo local con alta capacidad de procesamiento y GPU para el entrenamiento de YOLOv8 . Se estima un coste de amortización de **400 €**.

Software: Python 3.10 con librerías (Pandas y YOLOv8 (Open Source)) . Coste: **0 €**.

Cloud/Almacenamiento: Servicios de alojamiento para el scraping continuo y almacenamiento de datos. Coste: **0 €**.

Categoría	Descripción	Coste
Personal	Equipo de desarrollo (5 ingenieros) encargado del scraping, modelos de ML/DL y documentación.	12.500€
Infraestructura	Uso de equipos locales con capacidad de procesamiento GPU para el entrenamiento de YOLOv8 y conectividad.	400 €

Software	Herramientas de código abierto (Python, OpenCV, YOLOv8, Git)	0 €
TOTAL	Presupuesto final del proyecto	12.900 €

4.5 Viabilidad

El análisis de viabilidad de **MADRIVE** confirma que el sistema es técnica y operativamente sostenible, integrando herramientas de visualización geográfica y procesamiento de datos:

- **Viabilidad Técnica:**
 - **Visualización Geográfica:** El uso de librerías de mapeo (como Leaflet o Google Maps API, integradas vía Flask) permite situar las cámaras de la DGT de forma exacta mediante las coordenadas latitud y longitud almacenadas en la base de datos MariaDB.
 - **Procesamiento de Imágenes con YOLO:** La implementación del modelo YOLO garantiza una detección de vehículos veloz, permitiendo que el sistema actualice el estado del mapa casi en tiempo real.
 - **Micro-framework Flask:** Actúa como el motor que sirve tanto la lógica de IA como la interfaz visual del mapa, asegurando una comunicación fluida entre el servidor y el cliente.
- **Viabilidad Operativa:**
 - **Fuentes de Información Públicas:** El sistema depende de imágenes de cámaras de la DGT y archivos de configuración JSON públicos, lo que asegura el suministro de datos sin costes externos.
 - **Interfaz Intuitiva:** Al representar los datos sobre un mapa, el usuario no necesita interpretar tablas de datos; simplemente observa visualmente qué tramos están congestionados.
- **Viabilidad Económica:**
 - **Software Open Source:** El uso de Python, MariaDB, HeidiSQL y Flask elimina gastos de licencias, haciendo que el proyecto sea viable con una inversión mínima de infraestructura.

4.6 Resultados del proyecto

Tras la fase de integración, se han obtenido los siguientes resultados tangibles:

- **Despliegue de Interfaz Geográfica Interactiva:**

- Se ha desarrollado un **módulo de visualización (Mapa)** que permite al usuario navegar por la ciudad de Madrid y localizar cada cámara monitorizada.
- Este mapa se alimenta dinámicamente de la tabla **zonas**, posicionando marcadores en las coordenadas geográficas reales.
- **Implementación de la Base de Datos Relacional:**
 - Se ha consolidado el esquema **pc2** en MariaDB, gestionando con éxito usuarios, zonas y el histórico de predicciones.
 - La base de datos está optimizada para que el mapa cargue los puntos de interés (POIs) de forma eficiente.
- **Eficacia en el Análisis de Tráfico:**
 - Se ha logrado automatizar el ciclo: "Captura de imagen -> Detección YOLO -> Predicción XGBoost -> Actualización en Base de Datos".
 - El sistema es capaz de traducir el conteo de vehículos en un estado visual sobre el mapa (por ejemplo, cambiando el color del marcador o mostrando el mensaje "Hay tráfico / No hay tráfico").
- **Conclusión del Prototipo:**
 - El resultado final es una aplicación web funcional donde el componente del **Mapa** actúa como el front-end principal, unificando toda la complejidad técnica de la IA en una experiencia de usuario sencilla y directa.

Capítulo 5. TECNOLOGÍAS USADAS

5.1 Python

El lenguaje principal utilizado en el desarrollo del proyecto ha sido **Python**, debido a su amplio ecosistema de bibliotecas orientadas al análisis de datos, machine learning y visión artificial. Python permite integrar fácilmente diferentes herramientas dentro de un mismo pipeline de procesamiento de datos.

5.2 YOLO (You Only Look Once)

Para la detección de vehículos en imágenes de tráfico se utilizó **YOLO**, un modelo de detección de objetos basado en redes neuronales convolucionales.

YOLO pertenece a la categoría de detectores *single-stage*, lo que significa que realiza la detección de objetos en una sola pasada de la red neuronal, permitiendo un equilibrio eficiente entre precisión y velocidad de procesamiento.

En este proyecto se empleó la implementación de YOLO disponible en la librería **Ultralytics**, que facilita la carga del modelo y su aplicación sobre imágenes. El modelo permite identificar automáticamente vehículos presentes en las imágenes de las cámaras de tráfico, generando como resultado el conteo de vehículos detectados en cada escena.

Este conteo constituye una de las variables principales utilizadas posteriormente para estimar el nivel de congestión del tráfico.

5.3 XGBoost

Para la predicción del estado del tráfico se utilizó **XGBoost (eXtreme Gradient Boosting)**, un algoritmo de aprendizaje por conjuntos basado en árboles de decisión.

XGBoost es una implementación optimizada del algoritmo **Gradient Boosting**, que construye múltiples árboles de decisión de forma secuencial para mejorar progresivamente la precisión del modelo. Este tipo de algoritmos es especialmente eficaz para trabajar con datos tabulares que combinan variables numéricas y categóricas.

En este proyecto se utilizó la clase **XGBClassifier**, que permite entrenar modelos de clasificación multiclase capaces de predecir el nivel de tráfico en una escala discreta.

Entre las principales ventajas de XGBoost se encuentran:

- Alta capacidad predictiva.
- Buena gestión de relaciones no lineales entre variables.
- Robustez frente al sobreajuste.
- Buen rendimiento con datasets estructurados.

5.4 Scikit-learn

La librería **Scikit-learn** se utilizó para las tareas de preprocesamiento de datos, división del dataset y evaluación del modelo.

En particular, se emplearon diferentes herramientas de esta biblioteca:

5.4.1 Normalización de datos

Para escalar las variables numéricas se utilizó **MinMaxScaler**, que transforma los valores de cada variable a un rango entre 0 y 1. Este proceso facilita el entrenamiento de los modelos de Machine Learning al evitar que variables con escalas diferentes dominen el aprendizaje.

5.4.2 Codificación de variables categóricas

Para convertir variables categóricas en valores numéricos se utilizó **LabelEncoder**, permitiendo que los modelos puedan procesar información como nombres de carreteras o identificadores de cámaras.

5.4.3 División del dataset

El conjunto de datos se dividió en datos de entrenamiento y de prueba mediante **train_test_split**, lo que permite evaluar el rendimiento del modelo sobre datos no utilizados durante el entrenamiento.

5.4.4 Evaluación del modelo

Para medir el rendimiento del modelo se utilizaron diferentes métricas de clasificación:

- **Balanced Accuracy.**
- **F1-score.**
- **Matriz de confusión.**
- **Classification report.**

Estas métricas permiten evaluar la precisión del modelo en la predicción de los distintos niveles de tráfico.

5.5 Pandas y NumPy

Para la manipulación y análisis de los datos se utilizaron las librerías **Pandas** y **NumPy**, ampliamente utilizadas en el ámbito del análisis de datos.

Pandas permite trabajar con estructuras de datos tabulares (DataFrames), facilitando tareas como:

- Limpieza de datos.
- Transformación de variables.
- Integración de datasets.

Por su parte, NumPy proporciona operaciones matemáticas eficientes sobre arrays numéricos, siendo utilizado como base para muchas operaciones de procesamiento de datos dentro del proyecto.

5.6 Entorno de Gestión de Datos: MariaDB y HeidiSQL

Para la implementación y administración de la capa de persistencia del proyecto **MADRIVE**, se han seleccionado herramientas que garantizan robustez y facilidad en la gestión de datos relacionales:

- **Motor de Base de Datos (MariaDB 10.4):** Se ha utilizado MariaDB como sistema de gestión de bases de datos (RDBMS) debido a su alto rendimiento y total compatibilidad con MySQL.
- **Gestor de Base de Datos (HeidiSQL 12.10):** Se ha empleado la interfaz gráfica **HeidiSQL** para el diseño, creación y mantenimiento de la estructura de la base de datos. Esta herramienta ha facilitado las siguientes tareas:
 - **Modelado de tablas:** Definición precisa de tipos de datos, como DECIMAL para coordenadas geográficas y ENUM para la gestión de roles de usuario.
 - **Ejecución de scripts:** Automatización de la creación de la base de datos pc2 y sus tablas mediante el volcado de archivos .sql.
 - **Monitorización:** Visualización en tiempo real de los datos capturados por los scripts de Python antes de ser servidos a la aplicación web.

Capítulo 6. DESCRIPCIÓN DEL PROBLEMA

El estado del tráfico en tramos urbanos concretos es altamente variable y depende de factores difíciles de anticipar (horarios pico, eventos locales, condiciones meteorológicas o incidencias puntuales). Esta variabilidad genera incertidumbre en los conductores, quienes carecen de una forma simple de saber, en un momento dado, si una zona estará congestionada o no.

Los datos relevantes para inferir dicha situación existen —cámaras públicas, registros web—, pero no se encuentran integrados ni transformados en un indicador comprensible. El problema a resolver consiste, por tanto, en disponer de un sistema que unifique información heterogénea y la convierta en una respuesta interpretada sobre el estado real del tráfico en un tramo urbano concreto, sirviendo como base para futuras capacidades predictivas.

Capítulo 7. ANÁLISIS Y DISEÑO DE LA SITUACIÓN

7.1 Análisis del sistema

El desarrollo del sistema parte de la necesidad de integrar diferentes fuentes de información relacionadas con el tráfico urbano y transformarlas en un indicador comprensible para el usuario. Para ello, se realizó un análisis previo de los componentes necesarios para capturar, procesar y visualizar los datos de tráfico en un entorno web.

Durante esta fase se identificaron tres elementos fundamentales del sistema:

- **Fuentes de datos externas**, principalmente cámaras públicas de tráfico y datos disponibles en páginas web oficiales.
- **Módulos de procesamiento**, encargados de transformar los datos brutos en información útil mediante técnicas de visión artificial y aprendizaje automático.
- **Interfaz de usuario**, que permite visualizar el estado del tráfico y generar predicciones de forma sencilla.

El análisis permitió determinar que la solución debía basarse en una arquitectura modular que facilitara la integración de estos componentes y permitiera la ampliación futura del sistema.

7.2 Requisitos del sistema

A partir del análisis realizado se definieron los principales requisitos funcionales y no funcionales del sistema.

Requisitos funcionales

- Obtener datos de tráfico desde fuentes públicas mediante técnicas de web scraping.
- Procesar imágenes de cámaras de tráfico para detectar y contar vehículos utilizando modelos de visión artificial.
- Clasificar el estado del tráfico en diferentes niveles de congestión.
- Permitir al usuario seleccionar una zona específica dentro del mapa de Madrid.
- Generar predicciones sobre el estado del tráfico en función de los datos históricos disponibles.
- Permitir el registro y autenticación de usuarios dentro de la aplicación.

- Almacenar en una base de datos las predicciones generadas y las zonas analizadas.

Requisitos no funcionales

- **Escalabilidad:** permitir la incorporación de nuevas cámaras o ciudades sin modificar la arquitectura principal.
- **Modularidad:** cada componente del sistema debe poder actualizarse de forma independiente.
- **Usabilidad:** la interfaz debe ser clara e intuitiva para cualquier usuario.
- **Eficiencia:** el procesamiento de datos debe realizarse en tiempos razonables para permitir análisis cercanos al tiempo real.

7.3 Diseño de la arquitectura del sistema

El sistema se diseñó siguiendo una arquitectura en tres capas que permite separar claramente las responsabilidades de cada componente:

Capa de adquisición de datos

Esta capa se encarga de recopilar la información desde diferentes fuentes externas. Incluye módulos de scraping que utilizan librerías como **Requests** para obtener datos desde páginas web públicas y APIs disponibles.

También se encarga de la descarga de imágenes procedentes de cámaras de tráfico, que posteriormente serán utilizadas en el proceso de detección de vehículos.

Capa de procesamiento y análisis

En esta capa se realiza el tratamiento de los datos obtenidos. Incluye diferentes procesos:

- **Limpieza y normalización de datos**
- **Extracción de características relevantes**
- **Detección de vehículos mediante YOLO**
- **Clasificación del estado del tráfico mediante modelos de Machine Learning**

Los resultados generados se transforman en indicadores de congestión que pueden ser interpretados fácilmente por el sistema y por el usuario final.

Capa de presentación

La capa de presentación corresponde a la aplicación web que permite la interacción con el usuario.

Esta interfaz incluye:

- Un **mapa interactivo de Madrid** donde se muestran las cámaras disponibles.
- Un sistema de **selección de cámaras**.
- La visualización del **estado actual del tráfico** en dichas zonas.
- La posibilidad de **generar predicciones personalizadas**.

Además, la aplicación incorpora funcionalidades de autenticación y gestión de usuarios.

7.4 Diseño de la base de datos

Para almacenar la información generada por el sistema se diseñó una base de datos estructurada que permite gestionar tanto datos estáticos como dinámicos.

Las principales entidades del sistema son:

Usuarios

- id_usuario
- username
- email
- password
- rol

Zonas o cámaras

- id_camara
- nombre
- latitud
- longitud
- descripcion

Predicciones

- id
- id_usuario
- id_camara
- carretera
- latitud
- longitud
- fecha_consulta_usuario
- fecha_prediccion
- nivel_ocupacion

Este diseño permite relacionar a cada usuario con las predicciones generadas sobre diferentes zonas del mapa.

7.5 Diseño del flujo de datos

El flujo de funcionamiento del sistema sigue las siguientes etapas:

1. Obtención de datos desde fuentes externas mediante scraping.
2. Descarga de imágenes de cámaras de tráfico.
3. Procesamiento de imágenes mediante el modelo YOLO para detectar vehículos.
4. Generación de un dataset estructurado con latitud, longitud, cámara analizada, fecha, hora, ubicación, número de vehículos y nivel de tráfico.
5. Aplicación de modelos de Machine Learning para clasificar el estado del tráfico.
6. Almacenamiento de resultados en la base de datos.
7. Visualización de la información en la interfaz web mediante el mapa interactivo.

Este flujo permite transformar datos no estructurados en información útil que puede ser utilizada para analizar y predecir el estado del tráfico.

Capítulo 8. IMPLEMENTACIÓN

8.1 Desarrollo del sistema

La implementación del sistema se llevó a cabo utilizando el lenguaje de programación **Python**, aprovechando su amplio ecosistema de librerías orientadas al análisis de datos, visión artificial y desarrollo web.

El desarrollo se estructuró en diferentes módulos independientes que corresponden con las capas definidas en el diseño del sistema.

8.2 Implementación del módulo de scraping

El primer componente implementado fue el módulo de adquisición de datos, encargado de obtener información desde fuentes públicas relacionadas con el tráfico.

Para ello se utilizó la librería de **Requests**, para realizar solicitudes HTTP a páginas web.

Este módulo permite acceder a páginas que contienen información sobre cámaras de tráfico y descargar las imágenes asociadas a cada ubicación.

Los datos obtenidos incluyen:

- Identificador de la cámara.
- Ubicación geográfica por latitud y longitud.
- URL de la imagen.
- Fecha y hora de captura.

Una vez extraídos, los datos se almacenan temporalmente en estructuras tabulares que posteriormente serán utilizadas en el proceso de análisis.

8.3 Implementación del modelo de detección de vehículos

Para el análisis de las imágenes de tráfico se utilizó el modelo de visión artificial **YOLO (You Only Look Once)**, el cual analiza cada imagen de las cámaras y genera:

- Número de vehículos detectados.
- Coordenadas de los objetos identificados (dato no guardado)

- Nivel estimado de ocupación de la vía

Esta información permite generar una medida objetiva de la densidad del tráfico en un momento determinado.

8.4 Implementación del modelo predictivo

Una vez obtenido el conteo de vehículos y los metadatos asociados (fecha, hora, ubicación), se construyó un dataset estructurado para entrenar un modelo de Machine Learning.

Para llegar a esto, el dataset se tuvo que normalizar para llevar todas las variables a la misma escala: para las variables continuas se utilizó un escalador de tipo **MinMax**, modificando las variables para que tuviesen un rango de 0 a 1. Para la carretera de la imagen, se dividió el número de la letra, y se empleó la librería de **get_dummies** para generar una variable booleana por tipo de carretera:

- carretera_letra_A.
- carretera_letra_M
- carretera_letra_N.

La fecha y la hora se dividieron en sus distintas partes:

- año.
- mes.
- día.
- hora.
- minuto.

Además, se empleó otro **get_dummies** para crear más variables booleanas representantes de la franja horaria a la que pertenecen las entradas, siendo estas:

- franja_horaria_mañana.
- franja_horaria_tarde.
- franja_horaria_noche.

Se utilizó un modelo basado en **Gradient Boosting**, que permite manejar eficientemente variables heterogéneas y detectar relaciones no lineales en los datos.

El modelo fue entrenado con más de **16000 registros**, permitiendo clasificar el estado del tráfico en una escala discreta:

- **0**: tráfico nulo.
- **1**: tráfico bajo.
- **2**: tráfico moderado.

- 3: tráfico elevado.

Los resultados obtenidos muestran que el modelo es capaz de identificar patrones de tráfico y generar predicciones razonablemente precisas con una precisión (**accuracy**) de aproximadamente 75%.

8.5 Entorno Web y API (Frontend y Backend)

La interfaz de usuario y la lógica de servidor se han desarrollado como una aplicación web completa, permitiendo al usuario interactuar con el sistema de predicción de forma intuitiva, visual y centralizada. La arquitectura se divide claramente en una capa de presentación (Frontend) y una interfaz de programación de aplicaciones (API Web/Backend).

8.5.1 Frontend: Interfaz de Usuario

Para la construcción del cliente web se han utilizado tecnologías estándar como HTML5, CSS3 y JavaScript (Vanilla), priorizando un diseño moderno (caracterizado por efectos glassmorphism y animaciones fluidas) y totalmente responsivo para adaptarse a diferentes resoluciones de pantalla.

El frontend se compone de dos vistas principales:

- Pantalla de Autenticación (login.html): Constituye la puerta de entrada segura a la plataforma MaDrive. Dispone de un formulario de inicio de sesión validado visualmente y preparado para comunicarse asíncronamente con el servidor.
- Panel de Monitorización (Dashboard /mapa.html): Es la vista principal y el núcleo operativo de la aplicación.

Cartografía Interactiva: Utiliza la librería Leaflet.js para renderizar mapas de alta calidad obtenidos mediante OpenStreetMap y CARTO.

Representación en Tiempo Real: Las cámaras de tráfico de la DGT se ubican en el mapa mediante marcadores con coordenadas reales. Además, el mapa traza la geometría exacta de las principales carreteras cubiertas (A-1 a A-6, M-40, M-50, etc.), consultando de forma dinámica la API geométrica de Overpass.

Interacción y Predicción: Desde esta interfaz, el usuario puede seleccionar una cámara concreta y consultar su captura visual en ese mismo instante. Al seleccionar una cámara, se despliega un panel lateral interactivo desde donde el usuario puede introducir una fecha y hora específicas para solicitar predicciones sobre el nivel de congestión al sistema inteligente.

8.5.2 Backend y API REST

El servidor web actúa como coordinador entre la interfaz gráfica y los modelos de simulación de Machine Learning. Se ha desarrollado empleando Python junto con el micronúcleo web Flask.

Las funciones principales de esta capa son:

- Gestión de Rutas y Sesiones: El sistema enruta a los usuarios y verifica el estado de sus sesiones en tiempo real. Si un usuario no autenticado intenta acceder al mapa interactivo, la comunicación es interceptada para ser redirigida automáticamente a la solicitud de credenciales.
- Servicio de Autenticación: Dispone de un endpoint API (/api/login) que recibe y procesa de forma segura mediante peticiones POST (JSON) la validación de administradores y usuarios en el sistema.
- Endpoint de Predicción con Inteligencia Artificial (/predicir): Tras la solicitud del usuario en el frontend, el backend emite una llamada a esta ruta inyectando un paquete de datos.
- Recibe la información serializada (fecha, hora, ubicación e identificador de la cámara).

Pasa esta información estructurada al módulo predict.py, donde los datos son normalizados y enviados a un modelo pre-entrenado de XGBoost y normalizados con la instancia del MinMaxScaler empleados en su entrenamiento, residentes en memoria RAM para acelerar su ejecución.

El sistema de ML devuelve una clasificación numérica (niveles del 0 al 3) de congestión.

El backend mapea este resultado puro en información contextual visual (Descripciones estandarizadas como "Tráfico Fluido", "Atasco" y colores de semáforo). El resultado final retorna en milisegundos a la interfaz web mediante un paquete JSON para su respectiva actualización en pantalla de cara al usuario.

8.6 Integración con la base de datos

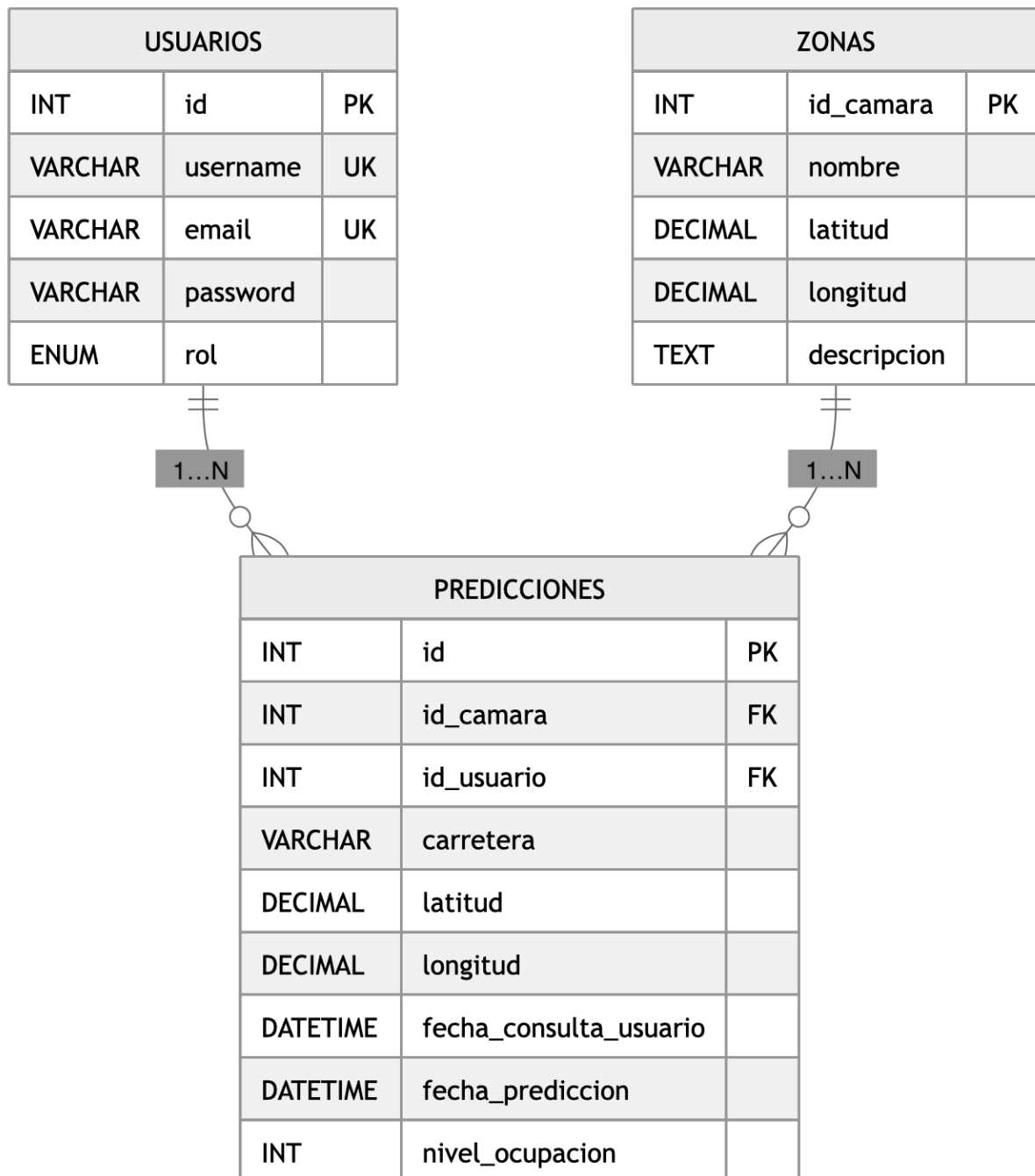
La aplicación se conecta a una base de datos (MariaDB) que permite almacenar la información generada por el sistema.

La base de datos gestiona:

- Usuarios registrados.
- Zonas o cámaras analizadas.
- Predicciones generadas.

Cada vez que un usuario realiza una predicción, esta queda almacenada junto con la fecha y la ubicación correspondiente, permitiendo consultar el historial de predicciones.

A continuación se muestra el diagrama de la base de datos:



Esta base de datos tiene como finalidad almacenar la información de los usuarios del sistema, las zonas o cámaras monitorizadas y las predicciones de ocupación consultadas por los usuarios. El modelo se ha estructurado en tres entidades principales: usuarios, zonas y predicciones. Esta organización permite separar

correctamente la información de autenticación y control de acceso, la información geográfica de las cámaras y los históricos de consultas o predicciones realizadas.

8.6.1 Tabla usuarios

La tabla usuarios almacena la información básica de las personas que acceden al sistema. Su objetivo es gestionar la identificación de cada usuario y permitir el control de permisos mediante roles. Cada registro representa un usuario único dentro de la aplicación.

Atributos de la tabla usuarios

id: campo de tipo entero (INT) que identifica de forma única a cada usuario. Es la clave primaria de la tabla (PRIMARY KEY) y su valor se genera automáticamente mediante AUTO_INCREMENT. Su función es servir como identificador interno del usuario dentro de la base de datos.

username: campo de tipo VARCHAR(50) que almacena el nombre de usuario. Tiene una restricción de unicidad (UNIQUE), lo que garantiza que no puedan existir dos usuarios con el mismo nombre. Este atributo permite identificar al usuario de manera pública dentro del sistema.

email: campo de tipo VARCHAR(100) que guarda el correo electrónico del usuario. También tiene una restricción de unicidad (UNIQUE), asegurando que cada dirección de correo solo pueda asociarse a una cuenta. Este atributo puede utilizarse para autenticación, recuperación de contraseña o comunicación con el usuario.

password: campo de tipo VARCHAR(255) destinado a almacenar la contraseña del usuario. El tamaño amplio del campo permite guardar contraseñas cifradas o hasheadas, lo cual es una práctica recomendable en términos de seguridad.

rol: campo de tipo ENUM('admin','usuario') que define el rol del usuario dentro del sistema. Su valor por defecto es 'usuario'. Este atributo permite diferenciar entre usuarios normales y administradores, facilitando así el control de acceso a determinadas funcionalidades.

Función de la tabla usuarios

Esta tabla centraliza la gestión de usuarios y permite implementar autenticación y autorización. Gracias al campo rol, el sistema puede distinguir entre perfiles con distintos niveles de privilegio. Asimismo, la unicidad de username y email evita duplicidades y mejora la integridad de los datos.

8.6.2 Tabla zonas

La tabla zonas almacena la información de las cámaras o ubicaciones geográficas monitorizadas por el sistema. Cada registro representa una zona específica sobre la que pueden realizarse predicciones de ocupación.

Atributos de la tabla zonas

id_camara: campo de tipo entero (INT) que identifica de forma única cada cámara o zona monitorizada. Es la clave primaria de la tabla. Este atributo actúa como identificador principal de cada ubicación.

nombre: campo de tipo VARCHAR(100) que almacena el nombre de la zona o cámara. Este dato facilita la identificación descriptiva de cada ubicación dentro del sistema.

latitud: campo de tipo DECIMAL(10,8) que guarda la coordenada geográfica de latitud de la zona. Se utiliza un tipo decimal para conservar precisión en la localización.

longitud: campo de tipo DECIMAL(11,8) que almacena la coordenada geográfica de longitud. Al igual que la latitud, este atributo permite posicionar con precisión la cámara o zona en un mapa.

descripcion: campo de tipo TEXT destinado a incluir información adicional sobre la zona, como características del entorno, detalles de localización o cualquier observación relevante.

Función de la tabla zonas

La tabla zonas permite gestionar las ubicaciones físicas asociadas a las cámaras del sistema. Su diseño resulta especialmente útil para aplicaciones que requieren representación geográfica, ya que incorpora coordenadas precisas de latitud y longitud. Además, su relación con la tabla predicciones hace posible conocer qué predicciones se han realizado sobre cada zona.

8.6.3 Tabla predicciones

La tabla predicciones es la entidad central del modelo, ya que registra cada consulta o predicción realizada en el sistema. En ella se enlaza la información del usuario que hace la consulta con la zona sobre la que se realiza la predicción, además de almacenar datos espaciales y temporales relacionados con dicha consulta.

Atributos de la tabla predicciones

id: campo de tipo entero (INT) que identifica de forma única cada predicción. Es la clave primaria de la tabla y se genera automáticamente mediante AUTO_INCREMENT.

id_camara: campo de tipo entero (INT) que actúa como clave foránea (FOREIGN KEY) y referencia al atributo id_camara de la tabla zonas. Este campo indica sobre qué zona o cámara se realiza la predicción.

id_usuario: campo de tipo entero (INT) que actúa como clave foránea y referencia al atributo id de la tabla usuarios. Permite saber qué usuario ha realizado la consulta o generado la predicción.

carretera: campo de tipo VARCHAR(50) que almacena el nombre o código de la carretera asociada a la predicción. Este dato añade un nivel descriptivo adicional a la ubicación.

latitud: campo de tipo DECIMAL(10,8) que guarda la latitud concreta asociada a la predicción. Aunque la zona ya tiene sus propias coordenadas, este atributo puede servir para registrar la ubicación exacta utilizada en el momento de la consulta.

longitud: campo de tipo DECIMAL(11,8) que almacena la longitud concreta vinculada a la predicción.

fecha_consulta_usuario: campo de tipo DATETIME que registra el momento en que el usuario realiza la consulta. Tiene como valor por defecto CURRENT_TIMESTAMP(), de modo que la fecha y hora se asignan automáticamente al insertar el registro.

fecha_prediccion: campo de tipo DATETIME que almacena la fecha y hora correspondientes a la predicción generada. Este atributo permite distinguir entre el momento de consulta y el instante temporal al que se refiere la predicción.

nivel_ocupacion: campo de tipo entero (INT) que representa el nivel de ocupación predicho para la zona consultada. Este valor puede interpretarse como una medida numérica de densidad, tráfico u ocupación según la lógica de la aplicación.

Función de la tabla predicciones

La tabla predicciones actúa como tabla transaccional o histórica del sistema. Su función es almacenar cada evento de consulta realizado por los usuarios, permitiendo mantener trazabilidad sobre quién consultó, qué zona consultó, cuándo lo hizo y cuál fue el resultado de la predicción. Gracias a ello, esta tabla puede utilizarse tanto para mostrar información al usuario como para realizar análisis posteriores o auditorías del uso del sistema.

8.7 Relaciones entre tablas

El modelo relacional presenta dos relaciones principales, ambas de tipo uno a muchos (1:N).

8.7.1 Relación entre usuarios y predicciones

Existe una relación **1:N** entre usuarios y predicciones. Esto significa que un usuario puede realizar muchas predicciones, pero cada predicción solo puede estar asociada a un único usuario. Esta relación se implementa mediante la clave foránea `id_usuario` en la tabla predicciones, que referencia al campo `id` de la tabla usuarios.

8.7.2 Relación entre zonas y predicciones

Existe también una relación **1:N** entre zonas y predicciones. Esto implica que una misma zona o cámara puede aparecer en múltiples predicciones, mientras que cada predicción solo se refiere a una única zona. Esta relación se implementa a través de la clave foránea `id_camara` en la tabla predicciones, que referencia al atributo `id_camara` de la tabla zonas.

8.7.3 Interpretación global del modelo

Desde un punto de vista funcional, el modelo refleja que los usuarios interactúan con el sistema consultando predicciones sobre determinadas zonas. La tabla predicciones sirve de nexo entre ambas entidades principales, registrando tanto el usuario que realiza la acción como la zona sobre la que recae la consulta. Este diseño evita redundancias, facilita la integridad referencial y permite ampliar el sistema en fases posteriores.

8.8 Claves y restricciones de integridad

La base de datos incorpora varias restricciones que garantizan la consistencia de la información almacenada.

Claves primarias:

La tabla usuarios utiliza `id` como clave primaria, la tabla zonas utiliza `id_camara` y la tabla predicciones utiliza `id`. Estas claves aseguran que cada registro sea único dentro de su tabla.

Claves únicas:

En la tabla usuarios, los campos `username` y `email` tienen restricción `UNIQUE`. Esto evita que existan dos usuarios con el mismo nombre de usuario o el mismo correo electrónico.

Claves foráneas:

La tabla predicciones incluye dos claves foráneas: id_camara, que referencia a zonas(id_camara), e id_usuario, que referencia a usuarios(id). Estas restricciones garantizan que no se puedan insertar predicciones asociadas a usuarios o zonas inexistentes.

Valores por defecto:

El campo rol en la tabla usuarios tiene como valor por defecto 'usuario', mientras que fecha_consulta_usuario en la tabla predicciones toma automáticamente la fecha y hora actuales. Esto mejora la automatización y reduce errores al insertar datos.

8.9 Justificación del diseño

La base de datos se ha diseñado de forma sencilla y organizada para que la información quede bien separada y sea fácil de gestionar. Por un lado, la tabla usuarios guarda los datos de acceso y el rol de cada persona; por otro, la tabla zonas recoge la información de las cámaras y su ubicación; y, por último, la tabla predicciones conecta ambas tablas y almacena cada consulta realizada junto con su resultado.

Con esta estructura se evitan datos repetidos, se mantiene mejor el orden en la base de datos y resulta más fácil hacer consultas, añadir información nueva o ampliar el sistema en el futuro.

8.10 Despliegue del sistema

Finalmente, el sistema fue preparado para su despliegue en un entorno web. Para ello se realizaron tareas de empaquetado de la aplicación, configuración del servidor y pruebas de funcionamiento.

El despliegue permite que la aplicación pueda ser utilizada desde cualquier navegador web, facilitando el acceso a la información de tráfico y a las predicciones generadas por el sistema.

Capítulo 9. CONCLUSIONES

El proyecto aún no ha terminado, por lo tanto se dejará el apartado de conclusiones hasta que se completen todos los hitos.

Capítulo 10. GLOSARIO

API (Application Programming Interface): conjunto de protocolos que permite la comunicación y el intercambio de datos entre diferentes aplicaciones de software de forma estructurada.

Ensemble Learning: técnica de aprendizaje automático (o aprendizaje por conjuntos) que combina múltiples modelos predictivos para obtener un resultado más preciso y robusto.

Escalador MinMax: método de preprocesamiento de datos que transforma y normaliza las variables numéricas continuas para que se ajusten a un rango entre 0 y 1.

ETL (Extract, Transform, Load): pipeline de ingeniería de datos que extrae información no estructurada, la transforma (estructura y limpia) y la carga en un sistema de destino. e visión artificial para detectar objetos a diferentes escalas dentro de una imagen.

Machine Learning: rama de la inteligencia artificial enfocada en entrenar modelos algorítmicos para que aprendan patrones y generen predicciones a partir de datos.

MAE (Mean Absolute Error): métrica de evaluación del rendimiento que calcula el promedio de los errores absolutos entre las predicciones del modelo y los valores reales.

Web Scraping: técnica informática automatizada utilizada para extraer de manera masiva datos e información directamente desde páginas web públicas.

XGBoost: algoritmo predictivo avanzado muy robusto y eficiente para manejar y clasificar datos estructurados heterogéneos.

YOLO (You Only Look Once): arquitectura de red neuronal de una sola etapa (*Single-Stage*) optimizada para la detección y clasificación de objetos en imágenes en tiempo real

Capítulo 11. BIBLIOGRAFÍA

Online Object Detector. Vladimir Iashin. [En línea].

[Citado el: 27 de octubre de 2025.]

<https://v-iashin.github.io/detector>.

Back-end code. Vladimir Iashin. [En línea].

[Citado el: 28 de octubre de 2025.]

<https://github.com/v-iashin/WebsiteYOLO>.

Información e incidencias de tráfico. DGT. [En línea] 2024.

[Citado el: 11 de octubre de 2025.]

<https://www.dgt.es/conoce-el-estado-del-trafico/camaras-de-trafico/?pag=1&prov=28&carr=>.

camaras.json. DGT. [En línea] 2024.

[Citado el: 26 de octubre de 2025.]

<https://www.dgt.es/.content/.assets/json/camaras.json>.

Requests. PyPI. [En línea].

[Citado el: 7 de octubre de 2025.]

<https://pypi.org/project/requests/>.

Capítulo 12. ANEXOS

No hay anexos adicionales necesarios para esta memoria.

[PÁGINA INTENCIONADAMENTE EN BLANCO]